05 — Making Inference Fast

Why generating text is limited by the speed of memory rather than of arithmetic, and the techniques that follow from that: caching, batching, quantisation and speculative decoding.

Reading time: about 55 minutes. You need: modules 00 to 04, and a rough idea of how a GPU's memory differs from its processors. You do not need: any experience of running a model in production.

Twenty-four tokens per second, and not one more

Take a model with 70 billion parameters, stored in 16-bit format. Its weights occupy 140 GB. Now imagine an accelerator with the memory speed of an NVIDIA H100, which moves about 3.35 TB per second, and enough memory to hold the whole model.

To generate a single token, the model must read every one of its weights once. That is 140/3,350 seconds, or 41.8 milliseconds. One token per 41.8 ms is 24 tokens per second, and this machine can never beat that, however well its code is written.

Meanwhile, its arithmetic capacity sits almost unused. During that 41.8 ms the chip could perform hundreds of times more calculations than generating a token requires. It spends nearly all of its time waiting for data to arrive from memory.

This one observation explains most of inference optimisation. This article works through it, then through the techniques it leads to.

What you will learn

By the end of this article, you will be able to:

- use arithmetic intensity to tell whether a computation is limited by memory or by arithmetic;
- implement a KV cache, explain why it is exact, and calculate how much memory it needs;
- choose how finely to quantise a model, and explain why outliers make it hard;
- explain speculative decoding, and why it leaves the output distribution exactly unchanged;
- report latency and throughput with the right metrics.

Sections 1 and 2 find the bottleneck. Sections 3 to 6 cover the techniques. Section 7 covers measurement. A recap and the exercise briefing follow at the end.

---

1. Two phases: reading the prompt, then writing the answer

A request to a language model happens in two phases that behave very differently.

Suppose a user sends a 1,000-token prompt and receives a 500-token answer. The first phase, called **prefill**, processes all 1,000 prompt tokens in a single forward pass. The matrix multiplications are large, and the GPU's processors stay busy. Prefill decides the **time to first token**: how long the user waits before anything appears.

The second phase, called **decode**, produces the answer one token at a time. It takes 500 steps, and each step processes a single new token. Yet each step still reads every weight in the model. Decode decides the **time per output token**: how fast the text appears once it starts.

Side by side, for that 1,000-token prompt and 500-token answer:

	prefill	decode
How many passes	1	500
New tokens per pass	1,000	1
Weights read per pass	all of them, once	all of them, once
Arithmetic per pass	1,000 tokens' worth	1 token's worth
Which metric it sets	time to first token	time per output token
What limits it	the chip's arithmetic	the speed of memory
How to make it faster	more FLOPs, or a shorter prompt	fewer bytes, or more users per pass

The last two rows are the whole of this article in miniature, and section 2 derives them. Mixing up the two phases is the most common mistake in inference questions. They have different bottlenecks, so they need different fixes.

---

2. Is the GPU calculating, or waiting for data?

To tell which phase is limited by what, we need one measurement. A **FLOP** is a single floating-point operation, such as one multiplication. The **arithmetic intensity** of a computation is the number of FLOPs it performs for each byte it moves between memory and processor.

First, what counts as one pass

"Forward pass" is used loosely, so here it means one precise thing: **one trip through every layer of the model, during which every weight is read from memory exactly once.** What varies is how many tokens ride along on that trip.

- In **prefill**, one pass carries all the prompt's tokens at once. A 1,000-token prompt is one pass with 1,000 tokens in it.
- In **decode**, one pass carries one new token per sequence. Generating 500 tokens is 500 passes.
- **Batching** adds sequences, not passes. Eight users decoding together is still one pass, now carrying eight tokens.

The weights are read once per pass in every one of those cases. That is the fact everything else rests on.

Where the 1 comes from

Take the smallest case: one decode step, one sequence, a model of N parameters stored in 16 bits.

The arithmetic. Every weight is used exactly once for this token, in one multiplication and one addition. That is 2 FLOPs per weight:

$$\text{FLOPs} = 2N$$

The bytes. Every weight is read from memory exactly once, and each one occupies 2 bytes:

$$\text{bytes} = 2N$$

The ratio. The N cancels, and so does the 2:

$$\text{arithmetic intensity} = \frac{2N}{2N} = 1 \text{ FLOP per byte}$$

The number does not depend on the model's size. A 7-billion-parameter model and a 70-billion-parameter one both sit at 1, because both do two FLOPs for every two bytes they read. It does depend on the format: the same model in 8-bit weights reads N bytes while still doing $2N$ FLOPs, giving an intensity of 2.

The general rule

Now put more tokens on the same trip. The weights are still read once, so the bytes do not change, while the arithmetic grows with every token carried:

$$\text{intensity} \approx \frac{2N \times \text{tokens in the pass}}{2N} = \text{tokens in the pass}$$

That is the rule worth remembering: **arithmetic intensity is roughly the number of tokens that share one reading of the weights.**

Situation	Tokens in the pass	FLOPs	Bytes	Intensity
decode, batch 1	1	$2N$	$2N$	1
decode, batch 8	8	$16N$	$2N$	8
decode, batch 64	64	$128N$	$2N$	64
prefill, 1,000-token prompt	1,000	$2{,}000N$	$2N$	1,000
prefill, 1,000 tokens, 8 users	8,000	$16{,}000N$	$2N$	8,000

Prefill is therefore the same computation as a very large decode batch, which is why it behaves completely differently.

The ridge point

Every chip has a **ridge point**: its peak FLOPs per second divided by its peak bytes per second. An H100 manages about 990 trillion 16-bit FLOPs per second of dense arithmetic, and its memory moves 3.35 TB per second. Both figures are for the SXM version, from NVIDIA's datasheet. Its ridge point is therefore 990/3.35, about 296 FLOPs per byte.

Below the ridge point, memory speed limits the work. Above it, arithmetic speed does. A decode step, at an intensity of 1, sits far below. It uses about 0.3% of the chip's arithmetic capacity.

What batching does

Now serve several users at once. The weights are still read once per step, but they are now used for every user in the batch. So the arithmetic grows with the batch size while the bytes read for the weights stay the same:

Batch size	Arithmetic intensity	Share of the H100 ridge point
1	1	0.3%
8	8	2.7%

Batch size	Arithmetic intensity	Share of the H100 ridge point
64	64	21.7%
256	256	86.6%
1,024	1,024	346% (now limited by arithmetic)

This is why serving systems work so hard to keep batches full. A larger batch uses capacity that would otherwise sit idle.

The table above makes one simplification, and it is worth undoing straight away. It counts only the weights, which every sequence in the batch shares. Each sequence also has its own key-value cache, described in section 3, and that is read in full on every step and shared with nobody. So the bytes grow with the batch even though the weight bytes do not.

For Llama-2-70B with 4,096 tokens of context, one sequence's cache is 1.34 GB, against 140 GB of weights:

Batch	Weight bytes	Cache bytes	Intensity	Compare with the ideal
1	140 GB	1.3 GB	1.0	1
8	140 GB	10.7 GB	7.4	8
32	140 GB	42.9 GB	24.5	32
128	140 GB	171.8 GB	57.5	128

Batching still helps, but it stops paying in proportion once the cache rivals the weights. At long context lengths this is the limit that bites first, which is the reason section 3 spends so long on making the cache smaller.

What this means for every other technique

Three consequences follow from this single measurement:

- Quantisation speeds up decoding because it reduces the number of bytes, not the number of FLOPs. Storing weights in fewer bits cuts the time per token in proportion.
- Batching raises throughput almost for free, as the table shows.
- Speculative decoding checks several tokens in one pass, using idle arithmetic to save memory reads.

The effect of quantisation on the 70B model from the opening is easy to compute:

Weight format	Time per token	Maximum tokens per second
16-bit	41.8 ms	24
8-bit	20.9 ms	48
4-bit	10.4 ms	96

> **In the interview:** open any inference question by asking whether the workload is limited by memory or by arithmetic. "Decoding at batch size 1 has an arithmetic intensity of about 1, far below the ridge point" is a strong first sentence.

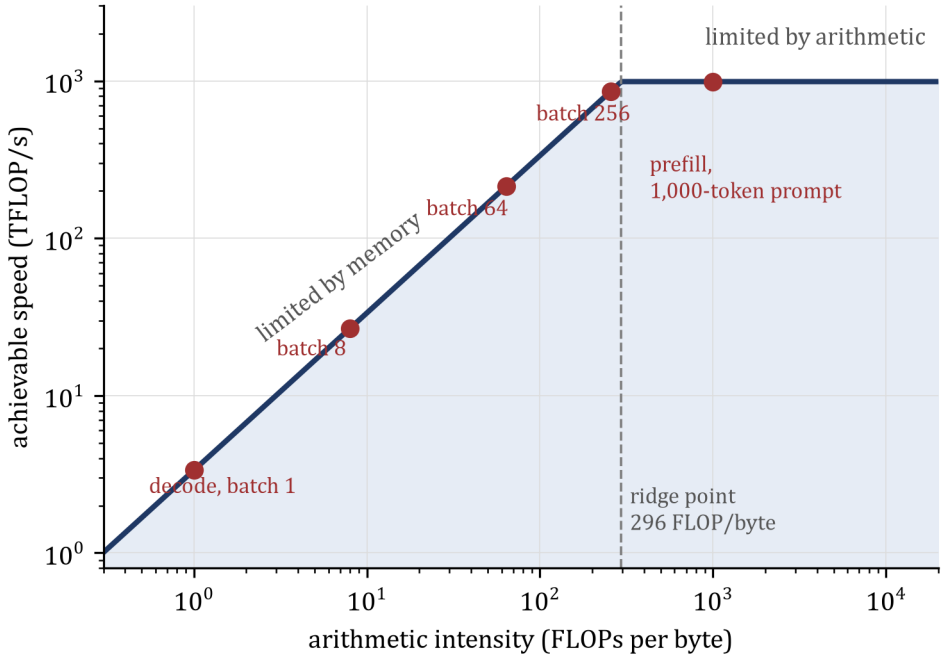


Figure 1: Each phase of inference placed against an H100's roofline. The sloped part is where memory speed decides, and the flat part is where arithmetic does. Decoding one token at a time sits almost three hundred times below the corner.

3. The KV cache

The problem it solves

Generating token 501 requires attention over tokens 1 to 500. Without a cache, the model recomputes the keys and values for all 500 earlier tokens at every step. But those keys and values have not changed since the previous step. Only the newest token is new.

The **KV cache** stores the keys and values from earlier steps, and computes them only for the new token. Generating T tokens then costs work in proportion to T rather than T^2.

What it looks like step by step

Take a three-token prompt and generate two tokens. Write t for the number of tokens the model has seen. Only the final row of this table changes shape from one step to the next:

Step	Tokens fed in	Rows in Q	Rows in K and V	Score matrix	Cache after the step
prefill	3	3	3	3×3	3 tokens
decode 1	1	1	4	1×4	4 tokens
decode 2	1	1	5	1×5	5 tokens

Two things follow, and both matter when you write the code.

The query has one row during decode while the keys have many, so **the score matrix is not square**. PyTorch's `is_causal=True` builds its mask from the top-left corner, which on a 1×5 matrix keeps only the first key. That is the trap the exercise warns about.

And once the cache is in use, **decode needs no causal mask at all**. The single query sits at the newest position, and every cached key is older, so there is nothing to hide. Masking returns only when padding puts fake tokens in the batch.

Why it is exact

The cache is not an approximation. In a model with a causal mask, each token can only attend to itself and to earlier tokens. So adding a new token cannot change any earlier key or value. The cached result must equal the full recomputation.

The exercise confirms it. Cached and uncached generation differed by at most 1.7×10^{-16}, which is rounding error. On a small test model, generating 96 tokens took 96 ms without the cache and 29 ms with it.

This also shows why caching would be wrong for a bidirectional model such as BERT. There, earlier tokens do attend to later ones, so a new token changes everything before it.

What it costs

The cache has to hold a key and a value for every layer, every token and every attention head:

$$\text{cache bytes} = 2 \times L \times s \times b \times n_{kv} \times d_{\text{head}} \times \text{bytes per number}$$

Here L is the number of layers, s the sequence length, b the batch size, n_{kv} the number of key–value heads, and d_{head} the size of each head. The leading 2 counts keys and values.

Take the shape of Llama-2-70B: 80 layers, and 64 attention heads of size 128. Suppose every head kept its own keys and values, as in standard multi-head attention. Then one 4,096-token sequence in 16-bit format needs $2 \times 80 \times 4{,}096 \times 1 \times 64 \times 128 \times 2$ bytes, which is 10.7 GB. A batch of 32 such sequences needs 344 GB, far more than the 140 GB of weights. At scale, the cache is the real memory limit.

Making the cache smaller

Several techniques shrink the cache:

- Multi-query attention (MQA) lets all query heads share a single key–value head.
- Grouped-query attention (GQA) is a compromise: groups of query heads share key–value heads. Llama-2-70B actually uses GQA with 8 key–value heads.
- Multi-head latent attention (MLA), from DeepSeek, stores a compressed low-rank version of the keys and values.
- Quantising the cache to 8 bits halves its size again.

For the same 70B shape, the three attention layouts compare like this:

Layout	Key–value heads	One sequence of 4,096 tokens	Batch of 32
MHA	64	10.7 GB	344 GB
GQA-8	8	1.34 GB	42.9 GB
MQA	1	0.17 GB	5.4 GB

A related technique, **PagedAttention** (used in the vLLM server), attacks wasted memory rather than cache size. It stores the cache in small fixed-size blocks, like pages in an operating system. A request no longer needs to reserve memory for its longest possible answer in advance.

4. Serving many users at once

A server rarely handles one request at a time. How it groups requests has a large effect on cost.

Static batching waits until every sequence in a batch has finished. One long answer keeps the whole batch waiting, while the finished slots sit empty. Four requests of different lengths, in a server with four slots, look like this. Each column is one decode step, and `..` is a wasted slot:

```
static batching
  slot 0 | A A A A A A A A
  slot 1 | B B B .. .. .. .. ..
  slot 2 | C C C C C .. .. ..
  slot 3 | D D .. .. .. .. .. ..
```

Eighteen of those 32 slots do useful work. The other 14 are paid for and wasted, because nothing new may start until A finishes.

Continuous batching removes finished sequences and adds new ones at every decode step. The same four requests, with E, F, G and H waiting in the queue:

```
continuous batching
  slot 0 | A A A A A A A A
  slot 1 | B B B E E E E E
  slot 2 | C C C C C F F F
  slot 3 | D D G G G G G H
```

No idle slots, and four extra requests served in the same wall-clock time. The charts are an illustration rather than a measurement. The Orca paper, which introduced the technique, reports up to 36 times the throughput of a static-batching server at the same latency, on a mix of request lengths. The vLLM server made it standard.

Chunked prefill splits a long prompt's prefill into pieces, and interleaves them with other users' decode steps. A single huge prompt then no longer freezes everyone else's output. It trades a little latency for one user against smoother latency for all.

---

5. Quantisation

Quantisation stores numbers in fewer bits, usually as integers with a shared scale factor.

A worked example

Take four weights: 0.1, −0.5, 2.0 and −1.2. We will store them as 8-bit signed integers, from −127 to 127. **Symmetric quantisation** picks a scale so that the largest magnitude maps to 127:

$$\Delta = \frac{\max_i |x_i|}{127}, \qquad q_i = \text{round}(x_i/\Delta), \qquad \hat{x}_i = \Delta\, q_i$$

Three symbols, in the order they are used. Δ is the scale, the real value that one integer step represents. q_i is the stored integer. The hat on $\hat{x}_i$ marks the number you get back after converting the integer to a real value again, which is close to x_i but rarely equal to it. The gap between x_i and $\hat{x}_i$ is the quantisation error.

Here $\Delta = 2.0/127 = 0.015748$. The integers are $q = (6, -32, 127, -76)$. Converting back gives 0.0945, −0.5039, 2.0 and −1.1969, all close to the originals.

Now add one large value, 60.0, to the same tensor. The scale becomes $60/127 = 0.4724$, and the integers become $(0, -1, 4, -3, 127)$. Converting back gives 0, −0.472, 1.890, −1.417 and 60.0. The original 0.1 has vanished entirely, and the others have lost most of their precision. One outlier spoiled the whole tensor.

Granularity: how many numbers share a scale

That example suggests the key decision: how many values share one scale factor.

- Per-tensor: one scale for the whole matrix. Cheapest, but one outlier affects everything.
- Per-channel: one scale for each row or column. Much more accurate when channels differ in size, which in transformers they do.
- Per-group: one scale for every group of, say, 128 weights. A compromise used by GPTQ and AWQ.

The exercise measures the difference on a matrix whose columns range in size from 0.1 to 10. The result depends heavily on how you measure the error:

Method	Overall relative error	Error in the worst column
per-tensor	1.48%	78.6%
per-channel	0.70%	1.0%

Measured over the whole matrix, per-channel looks only about twice as good. Measured column by column, per-tensor destroys the small columns almost completely. The overall error is dominated by the large columns, which per-tensor quantisation handles well. So always check the worst case, not only the average.

Bits matter as well. With per-channel scales, each extra bit roughly halves the error. The error was 12.7% at 4 bits, 2.9% at 6 bits, 0.70% at 8 bits and 0.043% at 12 bits.

Weights or activations?

Decoding is limited by memory, and the weights dominate what is read. So quantising only the weights captures most of the speed-up, while activations stay in 16-bit format. This is called **weight-only quantisation**, written W8A16 or W4A16. In the exercise, 8-bit weights changed a linear layer's output by only 0.39%.

It is worth being clear about where the integers stop. The weights are stored and moved as 8-bit integers, then converted back to 16-bit floats inside the kernel, right after they arrive. The matrix multiplication itself still runs in 16-bit arithmetic. So weight-only quantisation halves the bytes and changes the FLOPs not at all, which is exactly the right trade for a phase that sits far below the ridge point.

Quantising activations as well is harder, because of **outliers**. Dettmers et al. (2022) measured where they come from. Above roughly 6.7 billion parameters, a few feature dimensions start to carry values far larger than the rest, and they sit in the same positions across every token. The exercise plants one such channel. Per-tensor error jumps from 1.06% to 11.6%, while per-channel error stays at 0.51%.

The main responses to outliers are these:

- LLM.int8() keeps the outlier channels in 16-bit and quantises the rest.
- SmoothQuant moves the difficulty from activations to weights. It divides the activations by a per-channel factor and multiplies the weights by the same factor, which leaves the result unchanged.

- AWQ identifies the small fraction of weights that matter most, using activation sizes, and protects them.
- GPTQ quantises weights one column at a time, adjusting the remaining columns to compensate for the error so far.
- FP8, an 8-bit floating-point format, handles outliers better than 8-bit integers. It spends its bits on range rather than even spacing.

Two workflows are worth distinguishing. **Post-training quantisation** needs only a small calibration set and minutes of work. **Quantisation-aware training** fine-tunes the model with quantisation simulated during training. It recovers more quality, at far greater cost.

> **In the interview:** if asked how to quantise a model, start with weight-only quantisation, explain that decoding is memory-limited, and mention outliers as the reason activations are harder.

---

6. Speculative decoding

The idea

A small, fast **draft** model proposes several tokens, one after another. The large **target** model then checks all of them in a single forward pass. It can do so because checking positions in parallel is exactly what a transformer does during training. Correct tokens are kept. At the first rejected token, a replacement is sampled, and drafting starts again.

One round, with four drafted tokens, looks like this:

```
 draft model  | D1 D2 D3 D4                  four cheap passes, one token each
target model |           [ 1 pass ]     one expensive pass, checking all four at once
 verdict     |             ok ok ok  X     the first three match, the fourth does not
 kept        |          t1 t2 t3 t4'   t4' is resampled from the leftover distribution
```

Four tokens came out of a single pass of the large model. Had the fourth been accepted as well, the target's own prediction at that position would have been free, giving five. That is where the $\gamma + 1$ in the formula below comes from.

The remarkable part is that the output is distributed *exactly* as if the target model had generated it alone. Speculative decoding is not a trade of quality for speed. It is purely a speed-up.

The acceptance rule

Let q be the draft model's probabilities and p the target model's. This q is a distribution, not the quantised integer of section 5. The draft proposes a token x. The target accepts it with probability $\min(1,\ p(x)/q(x))$. If it rejects, it samples a replacement from the leftover distribution:

$$p'(x) = \frac{\max\left(0,\ p(x) - q(x)\right)}{\sum_{x'} \max\left(0,\ p(x') - q(x')\right)}$$

A worked example with three tokens

Suppose the target wants $p = (0.5, 0.3, 0.2)$ over three tokens A, B and C. The draft proposes from $q = (0.2, 0.3, 0.5)$. The acceptance probabilities are $\min(1, p/q) = (1, 1, 0.4)$. The leftover distribution is

$\max(0, p - q) = (0.3, 0, 0)$, which normalises to $(1, 0, 0)$: every rejection becomes A.

Now add up how often each token comes out:

- A is drafted 20% of the time and always accepted. It also appears whenever C is rejected. C is drafted 50% of the time and rejected 60% of the time, which adds 30%. In total: 50%.
- B is drafted 30% of the time and always accepted. In total: 30%.
- C is drafted 50% of the time and accepted 40% of the time. In total: 20%.

That is exactly p. Sampling 200,000 tokens this way gave 0.5019, 0.2989 and 0.1992. Tokens are accepted $0.2 + 0.3 + 0.2 = 70\%$ of the time, which equals $\sum_x \min(p(x), q(x))$.

Why it is exact, in three lines

A token x can come out in two ways: drafted and accepted, or sampled after a rejection. Add them.

Drafted and accepted has probability $q(x) \cdot \min(1, p(x)/q(x))$, and multiplying through the minimum gives $\min(q(x), p(x))$. A rejection happens with probability $\sum_{x'} \max(0, p(x') - q(x'))$, which is exactly the normaliser of p'. So the second route contributes $\max(0, p(x) - q(x))$, the numerator alone. Adding the two routes:

$$\min(p(x), q(x)) + \max(0, p(x) - q(x)) = p(x)$$

The identity holds whichever of $p(x)$ and $q(x)$ is larger. If $q(x) \geq p(x)$ the first term is $p(x)$ and the second is 0. If $q(x) < p(x)$ the terms are $q(x)$ and $p(x) - q(x)$. Either way the output distribution is the target's, exactly.

How much faster?

With acceptance rate α and γ drafted tokens per round, the expected number of tokens produced per target-model pass is:

$$\mathbb{E}[\text{tokens}] = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

Acceptance rate α	γ = 1	γ = 2	γ = 4	γ = 8
0.5	1.50	1.75	1.94	2.00
0.7	1.70	2.19	2.77	3.20
0.8	1.80	2.44	3.36	4.33
0.9	1.90	2.71	4.10	6.13

Returns shrink quickly as γ grows, while the draft model's cost keeps rising. So a good draft model, with a high acceptance rate, matters more than drafting many tokens.

Newer variants avoid a separate draft model. Medusa adds extra prediction heads to the target model. EAGLE drafts using the target model's internal features. Lookahead decoding drafts from patterns in text generated so far.

7. Measuring speed honestly

Four numbers describe inference performance:

- Time to first token measures how long a user waits before output starts. Prefill decides it.
- Time per output token measures how quickly text appears after that. Decode decides it.
- Throughput is the total number of tokens per second across all users. Costs scale with it.
- **Goodput** is the throughput that still meets the latency target. It is the right number for planning capacity.

Throughput and latency trade against each other through the batch size. A larger batch serves more users per second, but each user's tokens arrive more slowly. An honest benchmark therefore reports both, usually as a curve over batch sizes with the latency target drawn as a line.

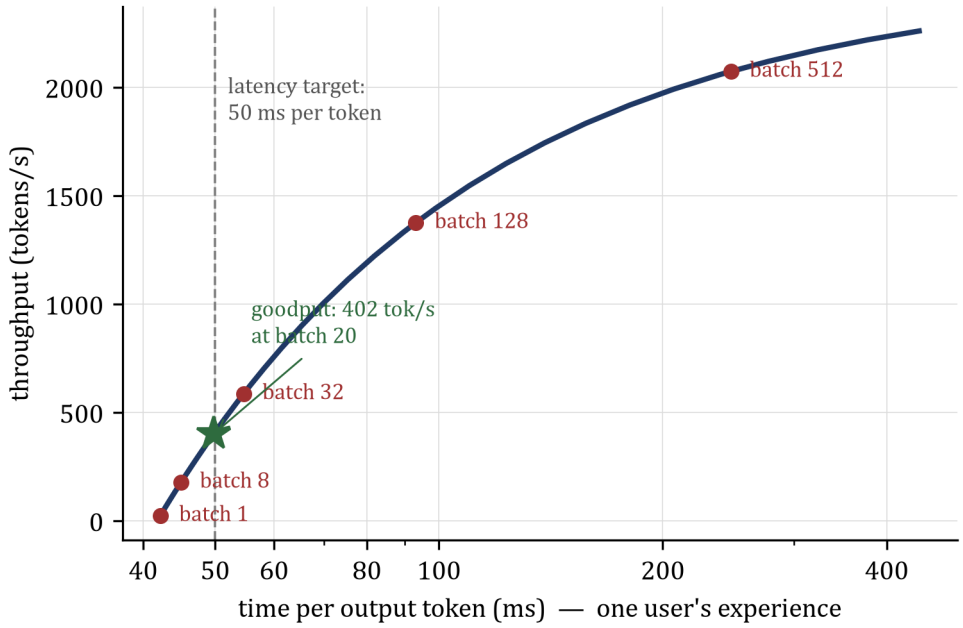


Figure 2: The trade-off for Llama-2-70B at 4,096 tokens of context, from the arithmetic of section 2 alone: time per token is whichever of the bytes and the FLOPs takes longer. Throughput keeps rising to the right, but only the part left of the latency target counts as goodput.

The curve is drawn from the two numbers section 2 used: bytes moved divided by memory bandwidth, against FLOPs divided by peak arithmetic, whichever is slower. Nothing else is needed to show the shape. Reading it at a 50 ms target gives a batch of about 20 and 402 tokens per second; allowing 200 ms would quadruple throughput and quadruple the wait.

What each technique actually buys

Technique	What it reduces	Which number improves	What it costs
KV cache	repeated computation	time per output token	memory, growing with batch and context
batching	weight reads per token	throughput	a longer wait per user
continuous batching	idle slots	throughput	a scheduler

Technique	What it reduces	Which number improves	What it costs
chunked prefill	one prompt blocking the rest	fairness of both metrics	slightly slower prefill
quantisation	bytes read per token	both latency numbers	accuracy, and outlier handling
GQA, MQA, MLA	cache bytes	memory, so larger batches	must be trained in
PagedAttention	cache memory wasted on reservations	throughput	bookkeeping
speculative decoding	passes of the large model	time per output token	the draft model, and wasted drafts

---

Recap

- Decoding reads every weight for every token, so it is limited by memory speed, not arithmetic.
- One pass reads every weight once, whatever rides along. So arithmetic intensity is roughly the number of tokens sharing that one reading: 1 for a single decode step, 1,000 for a 1,000-token prefill.
- Arithmetic intensity compared with the ridge point shows which limit applies. Batching raises it, until each sequence's own cache becomes the thing being read.
- The KV cache is exact because of the causal mask. At scale, it takes more memory than the weights.
- Quantisation cuts bytes. Check the worst-case error, and beware outliers in activations.
- Speculative decoding produces exactly the target distribution, using a rejection rule.
- Report latency and throughput together, not one of them alone.

The exercise: measure, cache, quantise and speculate

What you will build. Four pieces of inference engineering, each checked numerically:

1. calculators for arithmetic intensity and the fastest possible generation speed;
2. a KV cache, proved exact against full recomputation, plus a cache-size calculator;
3. symmetric quantisation at three granularities, including the outlier experiment;
4. the speculative-decoding acceptance rule, with an empirical check of the output distribution.

Time and setup. Allow three to four hours. The setup is the same as for module 00.

How to run it. Open `05-inference-optimization/exercise.ipynb`, or run `python exercise.py` from inside the module folder.

How the checks work. They work as in module 00. Before you start, the first part prints its headings and then reports that it is not implemented:

```
H100 ridge point: 296 FLOP/byte

  batch    intensity   vs ridge  regime
[FAIL] part 1  -- NotImplementedError:
```

Step map. This table links each part to the section of the article it practises.

Part	You write	The idea it practises	Section
1	`arithmetic_intensity`, `min_tpot_seconds`	memory-limited decoding	2
2a	`CachedAttention.forward`, `generate_with_cache`	the KV cache and why it is exact	3
2b	`kv_cache_bytes`	what the cache costs	3
3	`quantize_symmetric`, `quantize_group`, `worst_channel_error`	granularity and outliers	5
4	`speculative_step`, `expected_tokens`, `acceptance_rate`	speculative decoding	6

One trap in part 2a deserves a warning. With a cache, the query covers one token but the keys cover the whole prefix, so the score matrix is not square. The option `is_causal=True` masks the wrong triangle of a non-square matrix, so build the mask yourself.

When you are done. Your cached generation matches full recomputation to within 10^{-10}. The per-channel quantiser beats the per-tensor one by a wide margin on the worst column. And your speculative sampler reproduces the target distribution. The check requires every probability to match within 0.006, and a correct implementation typically matches to about 0.001.

Several parts end with short written questions, such as why quantisation speeds up decoding without reducing arithmetic. Answer each in a sentence or two.

If you get stuck.

- For part 1, write out the FLOPs and bytes for one decode step before writing any code.
- For part 2, test with a prompt of one token first, then two, and compare against full recomputation after each step.
- For part 4, test `speculative_step` with fixed values of the random number `u` before sampling.
- If you are still stuck, `solution.py` has a complete reference.

Want more? The notebook ends with four stretch goals: a full multi-token speculative loop, a PagedAttention-style block table, an 8-bit KV cache, and a latency-versus-throughput curve.

Going deeper

Start here

- Chen (2022), *Transformer Inference Arithmetic* — https://kipp.ly/transformer-inference-arithmetic/. A short, practical walk through the memory and compute sums for real models. The best companion to sections 2 and 3.

The bottleneck

- Pope et al. (2022), *Efficiently Scaling Transformer Inference*. arXiv:2211.05102. The standard analytical treatment. Section 3 does the arithmetic in full.
- Williams, Waterman & Patterson (2009), *Roofline: An Insightful Visual Performance Model for Multicore Architectures*, Communications of the ACM 52(4). The origin of arithmetic intensity and the ridge point.

The KV cache and serving

- Shazeer (2019), *Fast Transformer Decoding: One Write-Head is All You Need*. arXiv:1911.02150. The paper that introduced multi-query attention.
- Ainslie et al. (2023), *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*, EMNLP. arXiv:2305.13245. Grouped-query attention, and how to convert an existing model to use it.
- Kwon et al. (2023), *Efficient Memory Management for Large Language Model Serving with PagedAttention*, SOSP. arXiv:2309.06180. The vLLM paper.
- Yu et al. (2022), *Orca: A Distributed Serving System for Transformer-Based Generative Models*, OSDI. The paper that introduced continuous batching.
- Agrawal et al. (2023), *SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills*. arXiv:2308.16369. Chunked prefill.
- DeepSeek-AI (2024), *DeepSeek-V2*. arXiv:2405.04434. Section 2.1 introduces multi-head latent attention.

Quantisation

- Nagel et al. (2021), *A White Paper on Neural Network Quantization*. arXiv:2106.08295. The best single reference on granularity, calibration and quantisation-aware training.
- Dettmers et al. (2022), *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale*, NeurIPS. arXiv:2208.07339. Documents the outlier features from section 5.
- Xiao et al. (2023), *SmoothQuant*, ICML. arXiv:2211.10438.
- Frantar et al. (2023), *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers*, ICLR. arXiv:2210.17323.
- Lin et al. (2024), *AWQ: Activation-aware Weight Quantization*, MLSys. arXiv:2306.00978.
- Micikevicius et al. (2022), *FP8 Formats for Deep Learning*. arXiv:2209.05433.

Speculative decoding

- Leviathan, Kalman & Matias (2023), *Fast Inference from Transformers via Speculative Decoding*, ICML. arXiv:2211.17192. Theorem 1 proves the output distribution is unchanged. Read it after section 6.
- Chen et al. (2023), *Accelerating Large Language Model Decoding with Speculative Sampling*. arXiv:2302.01318. The same idea, developed independently at DeepMind.
- Cai et al. (2024), *Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads*. arXiv:2401.10774.
- Li et al. (2024), *EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty*. arXiv:2401.15077.

Next: module 06 moves the bottleneck outside the model, to retrieving the right information.